Text-to-LoRA: Background, Class-Scale Recreation, and Extensions

Victoria Lee
Deep Learning Midterm Project
`Text-to-LoRA recreation (arXiv:2506.06105)`

Abstract

Foundation models are typically specialized with parameter-efficient fine-tuning such as Low-Rank Adaptation (LoRA), which still requires per-task datasets, training runs, and hyperparameter search. *Text-to-LoRA* (T2L) Charakorn et al. [2025] instead trains a hypernetwork that emits task-specific LoRA weights from a natural-language task description in a single forward pass. This midterm report summarizes the original paper, describes our method-faithful class-scale recreation on a 16 GB GPU (original code; not a clone of the public repository), and outlines extensions for the final project. We implement both reconstruction distillation of oracle LoRAs and end-to-end supervised fine-tuning (SFT) of the hypernetwork, with L/M/S architectural variants, Bias-HyperInit, z-scored reconstruction targets, and text conditioning via a frozen sentence encoder.

1 Introduction

Large language models (LLMs) exhibit broad capabilities but remain costly to specialize. The dominant practice is to freeze the base weights and learn a small adapter—most commonly LoRA Hu et al. [2022]—for each downstream task. This induces engineering overhead: curated data, repeated optimization, and limited transfer across related tasks.

Charakorn et al. Charakorn et al. [2025] ask whether many LoRA adapters share an underlying adaptation mechanism that a neural network can compress and, at test time, decode from language alone. Their answer is **Text-to-LoRA (T2L)**: a hypernetwork Ha et al. [2016] conditioned on a text embedding of the task description that generates LoRA matrices A and B for selected modules (query and value projections) across all layers in one batched forward pass.

Midterm goals. For this course project we (i) review the paper's motivation, method, and empirical claims; (ii) reimplement the core method at class scale with independent code; and (iii) propose concrete extensions for the final submission.

2 Background: The Original Paper

2.1 Problem setting

Given tasks $\mathcal{T} = \{t^1, \ldots, t^T\}$ with fine-tuning sets $\mathcal{D}^i$ and natural-language descriptions Z^i, standard LoRA solves

$$\Delta W^i = \arg\min_{\Delta W^i} \mathcal{L}_{\text{SFT}}(\mathcal{D}^i, \Psi, \Delta W^i), \tag{1}$$

where Ψ denotes frozen pretrained weights and $\Delta W = BA$ with low rank r. T2L instead learns hypernetwork parameters θ such that

$$\Delta W^i_{m,l} = h_\theta(\phi^i_{m,l}), \qquad \phi^i_{m,l} = \text{concat}\big[f(z^i),\, E[m],\, E[l]\big], \tag{2}$$

Preprint.

task text $\xrightarrow{f}$ CLS $\oplus$ layer/module emb (+ A/B, rank for M/S)
$\rightarrow$ residual MLP backbone $\rightarrow$ head $\rightarrow$ LoRA A, B
$\rightarrow$ inject into frozen LLM $\rightarrow$ L1 (recon) or CE (SFT)

Figure 1: Method sketch matching Charakorn et al. [2025], implemented in `t2l/model.py`.

where $f(z)$ is a text embedding (e.g., CLS from `gte-large-en-v1.5`), and $E[m]$, $E[l]$ are learned embeddings for module type and layer index.

2.2 Architectures (L / M / S)

Most hypernetwork capacity sits in the output head. The paper proposes three inductive biases (Fig. 2 / App. F in Charakorn et al. [2025]):

- **L**: head emits both A and B ($2 \cdot r \cdot d$ outputs).
- **M**: shared head emits either A or B, selected by an A/B embedding ($r \cdot d$).
- **S**: head emits one rank vector at a time, with A/B and rank embeddings (d).

All variants share a backbone: task encoder $\rightarrow$ mixer MLP $\rightarrow$ residual MLP blocks $\rightarrow$ final MLP, with Bias-HyperInit Beck et al. [2023] for stable training (A-like uniform bias, B zeros; scaled for M/S).

2.3 Two training regimes

Reconstruction. Given a library of pretrained LoRAs Ω, minimize elementwise L1 between $h_\theta(\phi)$ and target adapters. The paper predicts *z-scored* weight entries and denormalizes at inference. Reconstruction recovers oracle benchmark LoRAs (Table 1) but struggles at zero-shot generalization when similar tasks have distant adapters in weight space (Table 6, App. D).

Supervised fine-tuning (SFT). Optimize θ end-to-end with language-modeling loss through the frozen LLM plus generated LoRA (Eq. 5), sampling a description per example. SFT on hundreds of Super-NaturalInstructions (SNI) Wang et al. [2022] tasks yields zero-shot adapters that beat multi-task LoRA and approach task-specific oracles on several benches (Table 2).

2.4 Key empirical takeaways

1. Lossy compression of many LoRAs is possible while retaining much of oracle accuracy.
2. Zero-shot generation requires *text* conditioning and benefits strongly from SFT over pure reconstruction.
3. Performance scales with number of training tasks *when compute is scaled accordingly*.
4. Generated adapters cluster by task semantics in activation space (t-SNE); description alignment matters (Table 5).

3 Class-Scale Recreation

3.1 Design constraints and fidelity

We target a **16 GB consumer GPU** and a midterm timeline, so we deliberately downscale compute while preserving the method: hypernetwork $\rightarrow$ LoRA A/B from text; L/M/S heads; Bias-HyperInit; z-scored reconstruction; SFT through injected LoRA; gte text encoder; q_proj/v_proj, rank 8.

Code is **original** (package `t2l/`, single CLI `run.py`), not a restructuring of SakanaAI/text-to-lora.

Table 1: Paper setup vs. our class-scale recreation.

	Paper	This project
Base LLM	Mistral-7B-Instruct	Qwen2.5-1.5B-Instruct
Embedder	gte-large-en-v1.5	same
Default variant	M (reported)	M
Oracle tasks	9 benches	6 (BoolQ, ARC-E, PIQA, . . .)
SFT corpus	479 SNI tasks	6 multi-task slots (synthetic default; HF/SNI optional)
SFT steps	up to $\sim$1M	30k default
Hardware	H100-class	16 GB GPU
Descriptions	GPT-4o mini	templates + paraphrases

3.2 Implementation overview

Hypernetwork (`t2l/model.py`). `HyperModulator` implements L/M/S. `TaskEmbedder` wraps a frozen gte encoder. `DynamicLoRALinear` replaces target linears so generated A, B remain on the autograd graph during SFT (buffers would block gradients).

Pipeline (`run.py`).

1. `descriptions` — build description banks.
2. `oracles` — train per-task PEFT LoRAs; pack A/B tensors.
3. `recon` — distill z-scored oracles with text (or one-hot) conditioning; early-stop on L1.
4. `sft` — end-to-end CE with bf16 + gradient checkpointing.
5. `multitask` / `eval` — baselines, zero-shot T2L, description ablation, t-SNE of task-encoder activations.

Memory recipe for 16 GB. Frozen base, bf16, gradient checkpointing, batch size 1 with gradient accumulation for SFT; reconstruction does not backprop through the LLM and is comparatively cheap.

3.3 Experimental protocol (midterm)

We evaluate on a subset of the paper's QA benches (default: BoolQ, ARC-Easy, PIQA; $N \leq 200$ examples for quick iteration) and report:

- **Reconstruction recovery**: recon-T2L accuracy vs. task-specific oracle LoRAs (Table 1 analog).
- **Zero-shot SFT-T2L** vs. base model and multi-task LoRA (Table 2 analog).
- **Description ablation**: aligned vs. random / cross-task text (Table 5 analog).
- **Qualitative analysis**: t-SNE clustering of encoder activations by task.

Absolute numbers are not expected to match the 7B / 479-task paper; we look for *qualitative* agreement (recon $\approx$ oracle; SFT $>$ base; sensitivity to description alignment).

3.4 Limitations of the recreation

- Smaller base model and far fewer SFT tasks/steps reduce zero-shot headroom.
- Default SFT corpus is synthetic for offline reliability; real HF classifiers or SNI require env flags (`T2L_USE_HF_TASKS`, `T2L_DOWNLOAD_SNI`).
- Template paraphrases are weaker than GPT-4o-mini description banks.
- Full Arrow Routing / Hyperdecoder baselines are omitted (noted only in README).

4 Possible Extensions (Final Project)

We prioritize extensions that are feasible on 16–24 GB GPUs yet scientifically meaningful relative to the paper.

4.1 Extension A: Scale task diversity under fixed compute

Reproduce the paper's finding that more tasks help *only* when steps scale (Table 3 / App. C): sweep $\{8, 16, 32\}$ real HF classification / SNI tasks at fixed vs. proportional step budgets; plot zero-shot bench average. **Deliverable:** scaling curves + short analysis of under-training vs. diversity.

4.2 Extension B: Reconstruction in ΔW space

App. K notes that $A\|B$ permutations break weight-space similarity, while flattened $\Delta W{=}BA$ correlates better with task embedding similarity. Train a recon head on BA (or SVD-aligned A, B) and compare zero-shot SFT-initialized-from-recon vs. scratch SFT. **Hypothesis:** ΔW-space recon improves transfer of the reconstruction library into the SFT regime.

4.3 Extension C: Steerable / compositional descriptions

Figure 4 in the paper shows answer-style steerability from wording alone. Build a controlled eval: same question, descriptions that request (i) short answer, (ii) chain-of-thought, (iii) wrong domain. Measure accuracy *and* format compliance. Optionally add a lightweight description rewriter (small local LM) before $f(z)$.

4.4 Extension D: Cross-model transfer

Paper limitation: whether T2L trained on a small sibling transfers within a family. Train hypernet on Qwen2.5-0.5B or 1.5B, evaluate generated LoRAs on 3B (same module layout) with frozen hypernet or a linear head remapping. **Deliverable:** transfer matrix across sizes.

4.5 Extension E: Beyond q/v LoRA

Replace or augment targets with MLP adapters / DoRA / rank patterns, or generate *activation* modulators instead of weights (discussion in Charakorn et al. [2025]). Even a single alternative output space (e.g., gate vectors per layer) would differentiate the final project from a pure replication.

4.6 Recommended final-project path

Primary: Extensions A + C (scaling + steerability) on real HF/SNI data with full eval tables and t-SNE.
Stretch: B or D if compute allows.
This yields a clear narrative: *faithful midterm recreation → final project probes scaling, language control, and/or transfer beyond the original experimental budget*.

5 Conclusion

Text-to-LoRA reframes adaptation as amortized inference over a hypernetwork conditioned on language. Our midterm recreation implements the core architectures and both training regimes at class scale on Qwen2.5-1.5B with original code suitable for a 16 GB GPU. The final project will move from replication toward analysis—especially task scaling, description steerability, and optionally ΔW-space reconstruction or cross-size transfer.

Acknowledgments and Disclosure of Funding

Course midterm project report for Deep Learning. Method based on Charakorn et al., "Text-to-LoRA: Instant Transformer Adaption," 2025.

References

M. Beck, et al. A Survey of Meta-Reinforcement Learning. *arXiv:2301.08028*, 2023. Bias-HyperInit as used by Charakorn et al. Charakorn et al. [2025].

R. Charakorn, E. Cetin, Y. Tang, and R. T. Lange. Text-to-LoRA: Instant Transformer Adaption. *arXiv:2506.06105*, 2025. `https://arxiv.org/abs/2506.06105`.

D. Ha, A. Dai, and Q. V. Le. HyperNetworks. *ICLR*, 2017. `https://arxiv.org/abs/1609.09106`.

E. J. Hu, et al. LoRA: Low-Rank Adaptation of Large Language Models. *ICLR*, 2022.

Y. Wang, et al. Super-NaturalInstructions: Generalization via Declarative Instructions on 1600+ NLP Tasks. *EMNLP*, 2022.